

Program Analysis 2025-26

Lecture #8

Sufficient Incorrectness Logic



UNIVERSITÀ
DI PISA

The taxonomy

	Forward	Backward
Over	$\{\text{HL}\} \llbracket c \rrbracket P \subseteq Q$	$(\text{NC}) \llbracket c \rrbracket^{op} Q \subseteq P$
Under	$\llbracket \text{IL} \rrbracket \llbracket c \rrbracket P \supseteq Q$	? Lecture #08

The taxonomy

	Forward	Backward
Over	$\{\text{HL}\} \llbracket c \rrbracket P \subseteq Q$	$(\text{NC}) \llbracket c \rrbracket^{op} Q \subseteq P$
Under	$\llbracket \text{IL} \rrbracket \llbracket c \rrbracket P \supseteq Q$	$\langle \text{SIL} \rangle \llbracket c \rrbracket^{op} Q \supseteq P$ sufficient incorrectness logic exposes sources of errors

**Different logics for different
purposes!**

Hoare Logic

$$\{P\} c \{Q\}$$

validity: $\llbracket c \rrbracket P \subseteq Q$

$$\forall \sigma \in P . \forall \delta \in \llbracket c \rrbracket \sigma . \delta \in Q$$

can prove the absence of bugs
(any execution of c from P is correct)

Example: HL

$\{x \leq 0, y = 1\}$

while $(x \leq 5)$ do $x := x + y;$

$\{x = 6\}$ ✓

Example: HL

$\{x \leq 0\}$

while $(x \leq 5)$ do $x := x + y$;

$\{x = 6\}$ $\otimes$ $[x \mapsto 8, y \mapsto 8]$ is also reachable

Example: HL

$\{x \leq 0\}$

while $(x \leq 5)$ do $x := x + y$;

$\{x \geq 6\}$ ✓

Example: HL

$\{x \leq 0\}$

while $(x \leq 5)$ do $x := x + y$;

$\{x \geq 0\}$ ✓

Necessary condition

$$(P) \ c \ (Q)$$

$$\text{validity: } P \supseteq \llbracket \overleftarrow{c} \rrbracket Q$$

$$\forall \delta \in Q. \forall \sigma \in \llbracket \overleftarrow{c} \rrbracket \delta. \sigma \in P$$

express necessary conditions for correctness
(any execution of c from outside P is incorrect)

Example: NC

$$(x \leq 6 \wedge y = 6 - x)$$

while $(x \leq 5)$ do $x := x + y$;

$$(x = 6) \quad \times$$

Example: NC

$$(x \leq 6 \wedge \exists n . n \times y = 6 - x)$$

while $(x \leq 5)$ do $x := x + y$;

$$(x = 6) \quad \checkmark$$

Example: NC

$(x \leq 6)$

while $(x \leq 5)$ do $x := x + y$;

$(x = 6)$ ✓

Incorrectness Logic

$$[P] \ c \ [Q]$$

validity: $\llbracket c \rrbracket P \supseteq Q$

$$\forall \delta \in Q . \exists \sigma \in P . \delta \in \llbracket c \rrbracket \sigma$$

can prove the presence of bugs
(any error in Q is reachable executing c)

Example: IL

$[x \leq 0]$

while $(x \leq 5)$ do $x := x + y$;

$[x = 6] \otimes [x \mapsto 6, y \mapsto -1]$ is not reachable

Example: IL

$[x \leq 0]$

$[x \mapsto -4, y \mapsto 10]$

while $(x \leq 5)$ do $x := x + y$;

$[x = 6 \wedge y > 0]$ ✓

$[x \mapsto 6, y \mapsto 10]$

Sufficient incorrectness logic (SIL)

OOPSLA 2025

Revealing Sources of (Memory) Errors via Backward Analysis

FLAVIO ASCARI, University of Pisa, Italy
ROBERTO BRUNI, University of Pisa, Italy
ROBERTA GORI, University of Pisa, Italy
FRANCESCO LOGOZZO, Meta Platforms, USA

Sound over-approximation methods are effective for proving the absence of errors, but inevitably produce false alarms that can hamper programmers. In contrast, under-approximation methods focus on bug detection and are free from false alarms. In this work, we present two novel proof systems designed to locate the source of errors via backward under-approximation, namely Sufficient Incorrectness Logic (SIL) and its specialization for handling memory errors, called Separation SIL. The SIL proof system is minimal, sound and complete for Lisbon triples, enabling a detailed comparison of triple-based program logics across various dimensions, including negation, approximation, execution order, and analysis objectives. More importantly, SIL lays the foundation for our main technical contribution, by distilling the inference rules of Separation SIL, a sound and (relatively) complete proof system for automated backward reasoning in programs involving pointers and dynamic memory allocation. The completeness result for Separation SIL relies on a careful crafting of both the assertion language and the rules for atomic commands.

CCS Concepts: • **Theory of computation** → **Logic and verification**; *Proof theory*; *Hoare logic*; **Separation logic**; *Programming logic*.

Additional Key Words and Phrases: Sufficient Incorrectness Logic, Incorrectness Logic, Outcome Logic

ACM Reference Format:

Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. 2025. Revealing Sources of (Memory) Errors via Backward Analysis. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 127 (April 2025), 28 pages. <https://doi.org/10.1145/3720486>

1 Introduction

Formal methods aim to automate the improvement of software reliability and security. Notable success stories are, e.g., the Astrée static analyzer [Blanchet et al. 2003], the SLAM model checker [Ball and Rajamani 2001], the certified C compiler CompCert [Leroy 2009], VCC for safety properties verification [Cohen et al. 2009], and the Frama-C platform for the integration of many C code analyses [Baudin et al. 2021]. Despite that, effective program correctness methods struggle to reach mainstream adoption, mostly because they exploit over-approximation to handle decidability issues and false positives are seen as a distraction by expert programmers. Being free from false positives is possibly the reason why *under-approximation* approaches for bug-finding, such as testing and bounded model checking, are preferred in industrial applications. Incorrectness Logic (IL) [O’Hearn 2020] is a new program logic for bug-finding: *any error state found in the post can be produced by some input states that satisfy the pre*. However, IL triples are not able to characterize precisely *the input states that are responsible for a given error*. This is possibly rooted in the *forward* flavor of the under-approximation, which follows the ordinary direction of code execution.

Authors’ Contact Information: Flavio Ascari, University of Pisa, Pisa, Italy, flavio.ascari@phd.unipi.it; Roberto Bruni, University of Pisa, Pisa, Italy, bruni@di.unipi.it; Roberta Gori, University of Pisa, Pisa, Italy, roberta.gori@unipi.it; Francesco Logozzo, Meta Platforms, Seattle, USA, logozzo@meta.com.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/4-ART127

<https://doi.org/10.1145/3720486>

Proc. ACM Program. Lang., Vol. 9, No. OOPSLA1, Article 127. Publication date: April 2025.

“SIL can characterise the source of errors”



Sufficient Incorrectness Logic (SIL)

Given a specification Q of the possible errors

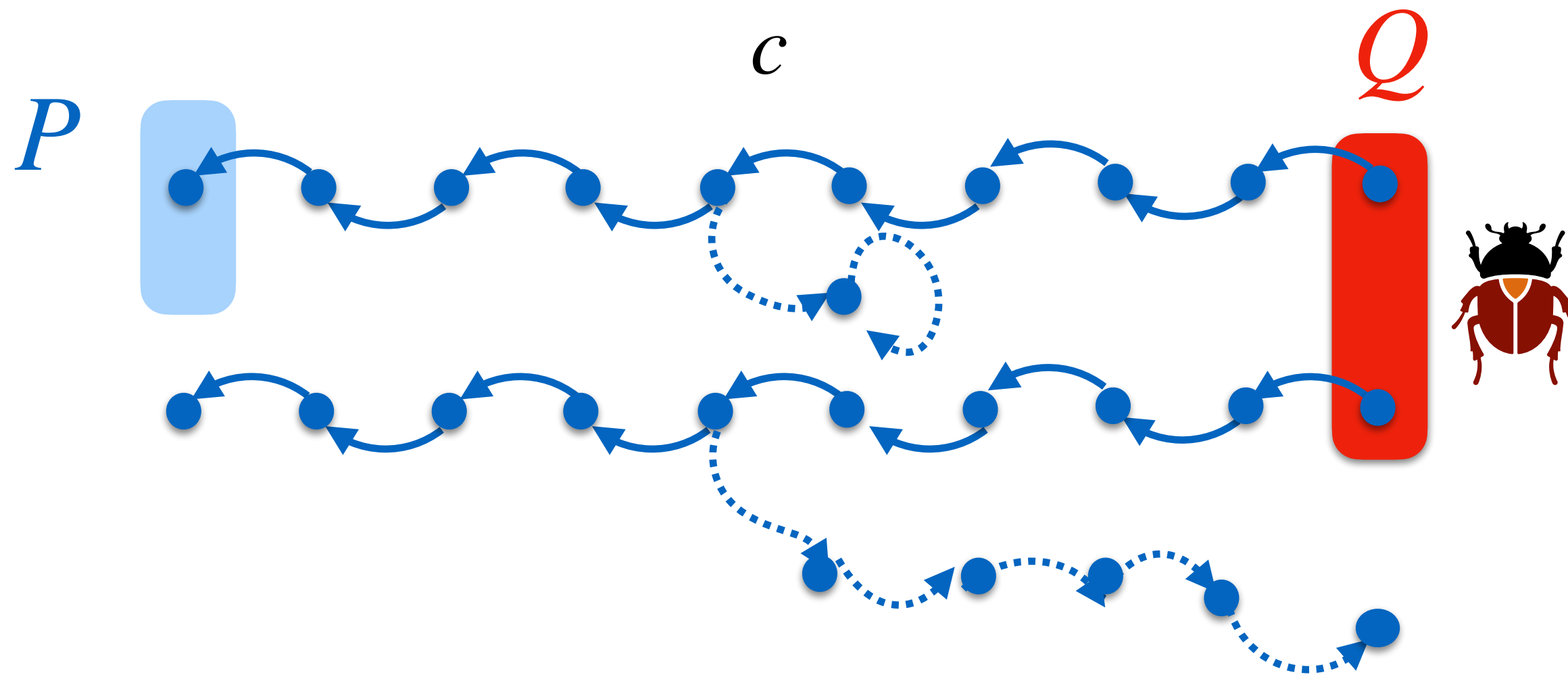
It is an under-approximation!

$\langle P \rangle c \langle Q \rangle$ is valid when

$$[[c]]^{op} Q \supseteq P$$

means

$$\forall \sigma \in P . \exists \delta \in Q . \delta \in [[c]]\sigma$$



A **backward under-approximation** logic to expose some initial states leading to errors

Sufficient Incorrectness Logic

$$\langle P \rangle \ c \ \langle Q \rangle$$

$$\text{validity: } P \subseteq \llbracket c \rrbracket^{op} Q$$

$$\forall \sigma \in P . \exists \delta \in Q . \delta \in \llbracket c \rrbracket \sigma$$

express sufficient conditions for incorrectness
(any state in P can lead within *er* : Q)

Example: SIL

$$\langle x \leq 6 \wedge \exists n . n \times y = 6 - x \rangle$$

while $(x \leq 5)$ do $x := x + y$;

$$\langle x = 6 \rangle \quad \checkmark$$

Example: SIL

$$\langle x \leq 6 \wedge y = 6 - x \rangle$$

while $(x \leq 5)$ do $x := x + y$;

$$\langle x = 6 \rangle \quad \checkmark$$

Example: SIL

$\langle x \leq 6 \rangle$ $[x \mapsto 5, y \mapsto -1]$ cannot reach the post

while $(x \leq 5)$ do $x := x + y$;

$\langle x = 6 \rangle$ $\otimes$

Bug reporting

Which errors should a tool report to programmers?

We do not want false positives but for the others?

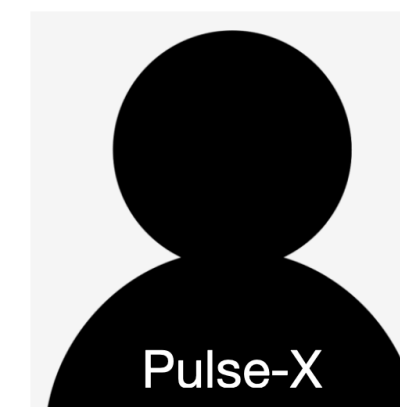
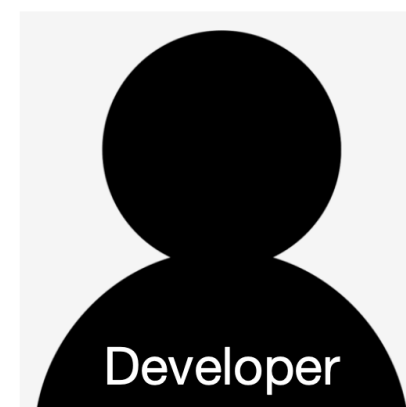
Should the tool report all of them?

```
int foo ( int * x)  
{ *x=32 }
```

Pulse (based on IL) would find

[x=null] foo(x) [er: x=null]

Should the tool report this?



“But I never call foo with null!”

“Which bugs shall I report then?”

Manifest errors

An error is **manifest** if it occurs **independently of the context**

It is therefore particularly interesting to point out to programmers

```
int main() {  
    printf("%d\n", f(5));  
    return 0;}  
// no f defined
```

Manifest errors cannot be characterised with IL

$$[\textit{true}] \ c \ [Q]$$

But they can be easily characterised with SIL

$$Q \text{ is a manifest error} \iff \langle \textit{true} \rangle \ c \ \langle Q \rangle \text{ is valid}$$

SIL Rules

The proof system favours backward analysis starting from the (error) postconditions



backward oriented

Hoare's axiom for assignment

$$\frac{}{\langle Q[a/x] \rangle x := a \langle Q \rangle} \text{ \langle Hoare \rangle }$$

$$\begin{aligned} &\langle y > 0 \rangle \quad x := y - 1 \quad \langle x \geq 0 \rangle \\ &\langle y \neq 43 \rangle \quad x := y - 1 \quad \langle x \neq 42 \rangle \end{aligned}$$

SIL Rules

The proof system favours backward analysis starting from the (error) postconditions



$$\frac{}{\langle Q \cap b \rangle b? \langle Q \rangle} \text{ \langle assume \rangle }$$

$$\begin{array}{ccccc} \langle \emptyset \rangle & (x > 0)? & \langle x = -42 \rangle \\ \langle x = 42 \rangle & (x > 0)? & \langle x = 42 \rangle \end{array}$$

SIL Rules

The proof system favours backward analysis starting from the (error) postconditions

Same conditions for both branches

backward oriented

$$\frac{\langle P_1 \rangle c_1 \langle Q \rangle \quad \langle P_2 \rangle c_2 \langle Q \rangle}{\langle P_1 \vee P_2 \rangle c_1 + c_2 \langle Q \rangle} \quad \langle \text{choice} \rangle$$

$$\langle y = 43 \vee y = 42 \rangle \quad (x := y - 1) + (x := y) \quad \langle x = 42 \rangle$$

$$\langle \text{true} \rangle = \langle y \neq 43 \vee y \neq 42 \rangle \quad (x := y - 1) + (x := y) \quad \langle x \neq 42 \rangle$$

$$\langle y \neq 43 \rangle \quad (x := y - 1) + (x := 42) \quad \langle x \neq 42 \rangle$$

SIL Rules

The proof system favours backward analysis starting from the (error) postconditions



backward oriented

Backward iteration starting from final state Q_0

$$\frac{\forall n \geq 0. \langle Q_{n+1} \rangle c \langle Q_n \rangle}{\langle \exists k. Q_k \rangle c^\star \langle Q_0 \rangle} \quad \langle \text{iter} \rangle$$

$$\langle x \leq 42 \rangle = \langle \dots \vee x = 41 \vee x = 42 \rangle \quad (x := x + 1)^\star \quad \langle x = 42 \rangle$$

SIL Rules

The proof system favours backward analysis starting from the (error) postconditions

SIL can drop disjuncts going backward:

$$\frac{}{\langle \text{false} \rangle \ c \ \langle Q \rangle} \quad \langle \text{empty} \rangle \qquad \frac{\langle P \vee R \rangle \ c \ \langle Q \rangle}{\langle P \rangle \ c \ \langle Q \rangle} \quad \langle \text{cons}' \rangle$$

$$\langle x = 41 \vee x = 42 \rangle \quad (x := x + 1)^{\star} \quad \langle x = 42 \rangle$$

Validity, soundness and completeness

A proof system for SIL

$\langle\langle Q \rangle\rangle$ stands for $\langle Q \rangle$

Core rules

$$\frac{}{\langle\langle [\overleftarrow{c}] Q \rangle\rangle \text{ c } \langle\langle Q \rangle\rangle} \text{ \texttt{atom} }$$

$$\frac{P \subseteq P' \quad \langle\langle P' \rangle\rangle \text{ r } \langle\langle Q' \rangle\rangle \quad Q' \subseteq Q}{\langle\langle P \rangle\rangle \text{ r } \langle\langle Q \rangle\rangle} \text{ \texttt{cons} }$$

$$\frac{\langle\langle P_1 \rangle\rangle \text{ r}_1 \langle\langle Q \rangle\rangle \quad \langle\langle P_2 \rangle\rangle \text{ r}_2 \langle\langle Q \rangle\rangle}{\langle\langle P_1 \cup P_2 \rangle\rangle \text{ r}_1 + \text{r}_2 \langle\langle Q \rangle\rangle} \text{ \texttt{choice} }$$

$$\frac{\langle\langle P \rangle\rangle \text{ r}_1 \langle\langle R \rangle\rangle \quad \langle\langle R \rangle\rangle \text{ r}_2 \langle\langle Q \rangle\rangle}{\langle\langle P \rangle\rangle \text{ r}_1; \text{r}_2 \langle\langle Q \rangle\rangle} \text{ \texttt{seq} }$$

$$\frac{\forall n \geq 0. \langle\langle Q_{n+1} \rangle\rangle \text{ r } \langle\langle Q_n \rangle\rangle}{\langle\langle \bigcup_{n \geq 0} Q_n \rangle\rangle \text{ r}^* \langle\langle Q_0 \rangle\rangle} \text{ \texttt{iter} }$$

Additional rules

$$\frac{}{\langle\langle \emptyset \rangle\rangle \text{ r } \langle\langle Q \rangle\rangle} \text{ \texttt{empty} }$$

$$\frac{\langle\langle P_1 \rangle\rangle \text{ r } \langle\langle Q_1 \rangle\rangle \quad \langle\langle P_2 \rangle\rangle \text{ r } \langle\langle Q_2 \rangle\rangle}{\langle\langle P_1 \cup P_2 \rangle\rangle \text{ r } \langle\langle Q_1 \cup Q_2 \rangle\rangle} \text{ \texttt{disj} }$$

$$\frac{}{\langle\langle Q \rangle\rangle \text{ r}^* \langle\langle Q \rangle\rangle} \text{ \texttt{iter0} }$$

$$\frac{\langle\langle P \rangle\rangle \text{ r}^*; \text{r } \langle\langle Q \rangle\rangle}{\langle\langle P \rangle\rangle \text{ r}^* \langle\langle Q \rangle\rangle} \text{ \texttt{unroll} }$$

$$\frac{\langle\langle P \rangle\rangle \text{ r}^*; \text{r } \langle\langle Q_1 \rangle\rangle}{\langle\langle P \cup Q_2 \rangle\rangle \text{ r}^* \langle\langle Q_1 \cup Q_2 \rangle\rangle} \text{ \texttt{unroll-split} }$$

Soundness and completeness

Validity of a **SIL** triple $\langle P \rangle \ c \ \langle Q \rangle$: $\llbracket c \rrbracket^{op} Q \supseteq P$

Th. [*Soundness*]

All provable SIL triples are valid

Th. [*Completeness*]

All valid triples are provable (using the core rules)

The taxonomy

The taxonomy

	Forward	Backward
Over	HL $\{P\} \ c \ \{Q\}$ $[[c]]P \subseteq Q$ $\forall \sigma \in P. \forall \delta \in [[c]]\sigma. \delta \in Q$	NC $(P) \ c \ (Q)$ $P \supseteq [[\overleftarrow{c}]]Q$ $\forall \delta \in Q. \forall \sigma \in [[\overleftarrow{c}]]\delta. \sigma \in P$
Under	IL $[P] \ c \ [Q]$ $[[c]]P \supseteq Q$ $\forall \delta \in Q. \exists \sigma \in P. \delta \in [[c]]\sigma$	SIL $\langle P \rangle \ c \ \langle Q \rangle$ $P \subseteq [[\overleftarrow{c}]]Q$ $\forall \sigma \in P. \exists \delta \in Q. \delta \in [[c]]\sigma$

Consequence rules

	Forward	Backward
Over	HL $\{P\} \ c \ \{Q\}$ $[[c]]P \subseteq Q$	NC $(P) \ c \ (Q)$ $P \supseteq [[\overleftarrow{c}]]Q$
Under	IL $[P] \ c \ [Q]$ $[[c]]P \supseteq Q$	SIL $\langle P \rangle \ c \ \langle Q \rangle$ $P \subseteq [[\overleftarrow{c}]]Q$

SIL vs IL

c_{42} :

```
if even (x) {  
    if odd(y) { z := 42; }  
}
```

Safe $z \neq 42$
e.g., $x := 1 / (42 - z)$

Given a specification of the possible errors

$Q \triangleq \{ z = 42 \}$

With **IL** one can prove

$[z=11] \ c_{42} \ [z=42 \wedge \textit{odd}(y) \wedge \textit{even}(x)]$

expressing that the postcondition is reachable

With **SIL** one can prove

$\langle z=11 \wedge \textit{odd}(y) \wedge \textit{even}(x) \rangle \ c_{42} \ \langle z=42 \rangle$

expressing a precondition that leads to error states

SIL vs HL

c_{42} :

$x := \text{nondet}();$

if *even* (x) {
 if *odd*(y) { $z := 42;$ }
}

Safe $z \neq 42$
e.g., $x := 1/(42 - z)$

Given a specification of the possible errors

$Q \triangleq \{z = 42\}$

With **HL** one can prove

$\{z=42\} \ c_{42} \ \{z=42\}$

With **SIL** one can prove

$\langle \text{odd}(y) \rangle \ c_{42} \ \langle z=42 \rangle$

expressing a precondition that leads to error states

SIL vs HL

If c is **deterministic** and **terminating**: **SIL** is equivalent to **HL**

$$\langle P \rangle \ c \ \langle Q \rangle \Leftrightarrow \{P\} \ c \ \{Q\}$$

Exercises

Validity

Which SIL triples are valid for any c and R ?

$\langle \text{false} \rangle \ c \ \langle R \rangle$



$\langle \text{true} \rangle \ c \ \langle \text{true} \rangle$



$\langle R \rangle \ c^\star \ \langle R \vee x = 0 \rangle$



$\langle wlp(c, R) \rangle \ c \ \langle R \rangle$



Soundness 1

Prove that rule $\langle \text{conj} \rangle$ is **unsound** for SIL

$$\frac{\langle P_1 \rangle \text{ c } \langle Q_1 \rangle \quad \langle P_2 \rangle \text{ c } \langle Q_2 \rangle}{\langle P_1 \wedge P_2 \rangle \text{ c } \langle Q_1 \wedge Q_2 \rangle} \quad \langle \text{conj} \rangle$$

Soundness 1

Prove that rule $\langle \text{conj} \rangle$ is **unsound** for SIL

$$\frac{\langle P_1 \rangle c \langle Q_1 \rangle \quad \langle P_2 \rangle c \langle Q_2 \rangle}{\langle P_1 \wedge P_2 \rangle c \langle Q_1 \wedge Q_2 \rangle} \quad \langle \text{conj} \rangle$$

Consider the triples $\langle x = 0 \rangle x := \text{nondet}() \langle x = 0 \rangle$

and $\langle x = 0 \rangle x := \text{nondet}() \langle x = 1 \rangle$

By rule $\langle \text{conj} \rangle$ we could derive $\langle x = 0 \rangle x := 1 \langle \text{false} \rangle$
which is not sound!

Soundness 2

Prove or disprove the validity of the following axiom in SIL

$$\langle P \rangle \ b? \ \langle P \wedge b \rangle$$

Soundness 2

Prove or disprove the validity of the following axiom in SIL

$$\langle P \rangle b? \langle P \wedge b \rangle$$

Consider the following triple: $\langle x \geq 0 \rangle \quad (x > 1)? \quad \langle x \geq 2 \rangle$

It is not valid, because from $x = 0$ we cannot reach $x \geq 2$